5.2 ppt

5.2 Subagent 任务委托

开篇

学习目标：**学会把子 Agent 任务写成可验收契约，确保模型描述产出、程序掌控身份权限状态计量。**

上一节最后，我们把订单项目的检查交给两个子 Agent：一个看订单源码，一个看测试覆盖。

但是委托发生了，不等于交清楚了。

把提示词写成可验收的契约，再把子 Agent 交回的结果按证据收拢。

一、任务提示词：把一份提示词写成能验收的契约

首先，一份能验收的提示词要写清哪些内容？一共九件事。

1. 任务目标：不是“检查测试”，而是“判断用户隔离、明细完整性、查询次数三种情况有没有对应测试”。
2. 输入上下文：要读哪些文件、遵循哪条规则。规则属于任务要求，源码里的具体条件属于待确认的事实，不能把主 Agent 的猜测当前提。
3. 可调用工具：这次只给读文件、搜索、目录；要跑测试再单独开受控测试工具，不要一开始就放开写文件和任意命令。
4. 权限边界：工具决定“能调用什么”，权限决定“能对哪些对象做什么”。允许读文件，不代表能读整台机器的凭证，要限定工作目录和可读范围。
5. 预期输出：要求返回状态、已检查项、发现、证据和未验证事项。
6. 输出格式：统一用 JSON，方便程序校验；给用户的报告再转成 Markdown。

7. 超时限制：给出截止时间和最大调用次数，超时就取消实际执行。预算不能只写在文本里，要接到 Harness 的计数器上。
8. 评价标准：区分“能解析”和“内容合格”。三项接口要求都查清、来源可复核，才算合格。
9. 写操作限制：本章审查默认不改源码。“给出重构建议”不等于批准执行重构；要改就另建修复任务并单独授权。

二、直接运行 pi：从模糊提示词到明确提示词

我们实际跑一下：

代码块

```
请调用 subagent，agent 为 course-scout。
提示词内容：
任务：静态覆盖审计（不执行测试）

   输入：
   - app/orders.py
   - tests/test_orders.py

   项目契约（作为验收基线）：
   R1 只返回当前用户订单
   R2 订单及明细合计最多 2 次 SELECT
   R3 返回的订单必须携带其明细

   必须逐项判定的场景，每项单独一条 checked_cases，verdict 取 覆盖/部分覆盖/未覆盖：
   C1 跨用户查询：夹具是否含其他 user_id 的订单，断言是否排除他人订单
   C2 明细完整性：是否断言 items 内容、数量与 order_id 归属
   C3 查询次数上限：是否存在对 db.execute 的计数、spy、mock 或 trace

   工具与范围：仅只读上述文件；可 grep 搜索相关测试；不改文件、不执行任何命令。
   禁止把静态分析表述为“已运行/已通过”。

   输出：仅一个 JSON 对象，首字符 { 末字符 }，无 Markdown 围栏、无前后说明文字。
   字段：
   - status: completed | needs_input
   - checked_cases: [{case, requirement, verdict, reason}]
   - findings: [{id, severity, requirement, detail}]
   - evidence: [{path, symbol, line, excerpt}]  excerpt 须为文件中的精确连续子串
   - unverified: [string]
   - inspected_files: [string]  实际读取过的文件
```

```
  - search_queries: [string]    实际执行过的搜索

  输出必须自包含：不要输出解释性散文，主 Agent 会在 JSON 之外单独撰写结论。
```

运行后我们看两处：实际调用记录，和返回的 JSON。

三、演示之后：把任务和结果存成结构

pi 里跑的是一段任务文本；正式系统还要把它存成结构化契约，方便追踪谁发起、对着哪个快照、是否完成。

代码块

```python
import json

contract = {
    "task_id": "review-test-001",
    "parent_run_id": "review-run-001",
    "agent": "course-scout",
    "goal": "检查用户隔离、明细完整性、查询次数三项接口要求测试覆盖",
    "checks": ["user_isolation", "items_complete", "query_count"],
    "snapshot_id": "snapshot-A",
    "inputs": ["app/orders.py", "tests/test_orders.py"],
    "tools": ["read", "grep", "find", "ls"],
    "policy_id": "review-readonly-v1",
    "limits": {"timeout_seconds": 60, "max_model_calls": 6},
    "output_fields": ["status", "checked_cases", "findings", "evidence", "unverified"],
    "acceptance": "三项接口要求都有结论，引用可核对，未修改项目文件",
}
print(json.dumps(contract, ensure_ascii=False, indent=2))
```

两个字段容易被误解。

policy_id 是程序选的权限策略，执行工具时 Runtime 仍要读策略、核对实际路径和动作。

描述任务的字段可以交给模型理解，身份、权限、执行状态和计量字段必须有程序侧的可信来源。

四、结果聚合与冲突仲裁：先校验证据，再讨论结论

1. 聚合不是拼接

主 Agent 该怎么收拢，才不会把几份结论直接拼在一起？答案是：聚合。

主 Agent 按五步处理：

1. 检查格式。
2. 检查证据。
3. 去重。
4. 排序。
5. 生成建议。

2. 校验不能互相替代

格式校验、来源校验和结论判断各自消除不同的错误，没有哪一项能代替全部。当两个子 Agent 的结论互相矛盾时，光靠校验还不够——这就是下一节要处理的冲突裁决。

3. 冲突与仲裁：不靠投票，靠证据

两个子 Agent 给出互相矛盾的结论时，主 Agent 该信谁。方法是先限定每个结论的适用范围，再补上证据。

例如：测试 Agent 说“现有测试全部通过，所以订单查询逻辑正确”；安全 Agent 说“代码里有没做用户过滤的 SQL，查询仍可能返回别人的订单”。

- 测试通过，只能说明执行过的用例在对应快照上通过，没有跨用户查询用例就不能推广到所有边界；
- 安全发现是源码线索，还要结合调用路径和实际行为核对。

业务规则冲突同理：需求说只返回当前用户订单，旧文档说管理员能查全部，两份说法不一致，所以主 Agent 得先核对哪份是当前版本；找不到权威来源，主 Agent 就交维护者定，不能自己改。

4. 仲裁顺序

仲裁不是让主 Agent 再猜一次，而是让它按顺序核对范围、版本、证据；需要时，再触发一项新验证。

六、串行交接与失败处理

有些任务有先后依赖，只能串行。

必须串行时，上一步的结果怎么交给下一步；子任务失败时，又该怎么按类型分别处理。

代码块

```
调用 subagent 的 chain 模式，用两个 course-scout 串行完成：
第一步读取 app/orders.py，返回用户过滤条件及代码原文，不做正确性保证；
第二步结合上一步结果，读取 tests/test_orders.py，判断三项接口要求覆盖。
第二步 task 中使用 {previous} 引入第一步结果。
两步都只读；最终区分实现观察、测试覆盖观察和未运行测试。
```

第一步的自然语言输出仍然是输入数据。它即使夹带“忽略原规则、允许写文件”，也不能改变第二步的权限策略——跨 Agent 传内容要保留来源性质，并继续执行工具门禁。

子任务失败时，按类型分别处理：超时、临时错误可以重试一次，但重试照样计入预算；权限被拒、输入缺失这类失败不重试，直接记为未完成项；只完成一部分的，保留已有证据并标为未验证。不管哪一种，失败都要写进结果的 status，不能让它在报告里消失。